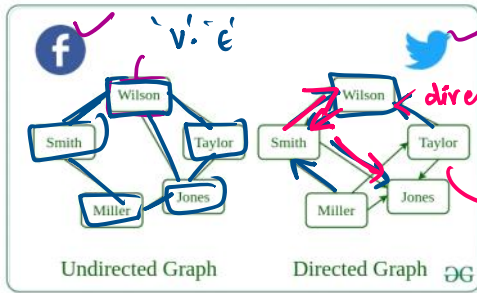


* what is Graph → non-linear data structure

Arrays, LL, stack, Queue ... * Edges

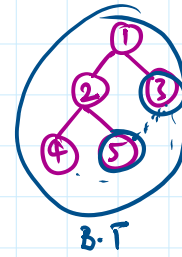
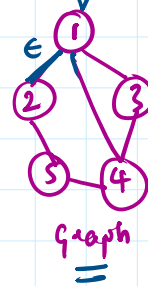
* vertices

* Nodes



direction added
real world ex:

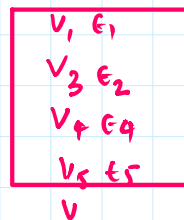
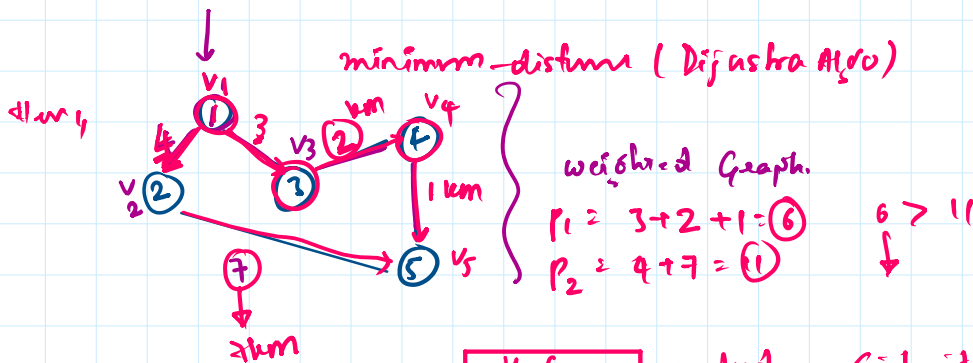
direct



Graphs → Directed (arrows/direction) } [direction]
→ Undirected. (don't arrows, edges)

Weighted → if edges contains values

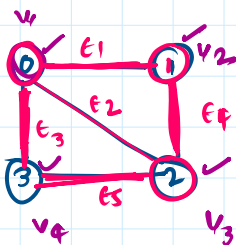
Un-weighted → if not —||—



undirected map (int, int) > shortest Distance;

Representation: Graph → Adjacent matrix (2D Array) $O(n^2)$
Adjacency List (LL)

AG

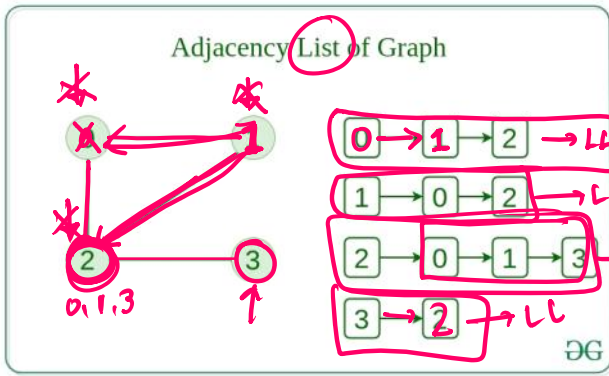


matrix

	v1	v2	v3	v4
v1	0	1	1	1
v2	1	0	1	0
v3	1	1	0	1
v4	1	0	1	0

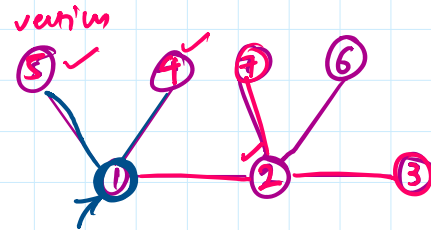
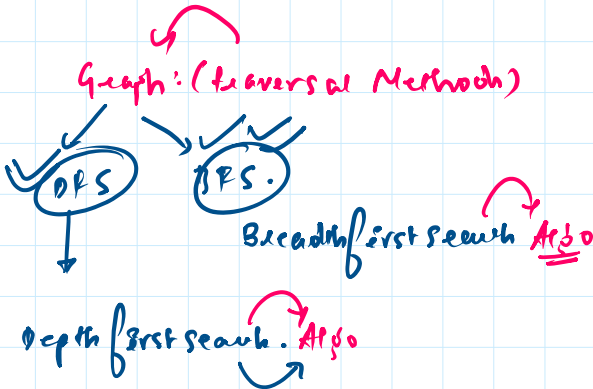
→ 2D array.
Adjacency matrix

Adjacency List of Graph



Ex. LL Rep. Graph.

- ① visiting vertex
- ② exploring vertex.



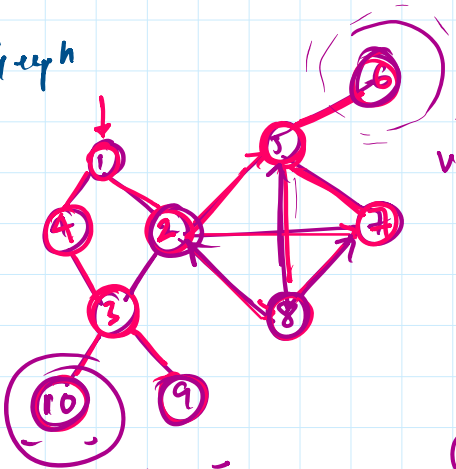
* BFS: 1, 2, 3, 5, 4, 7, 6.
 Queue D.A.
 breadth
 Nodes traversal

DPS: 1, 2, 4, 5, 3, 7, 6

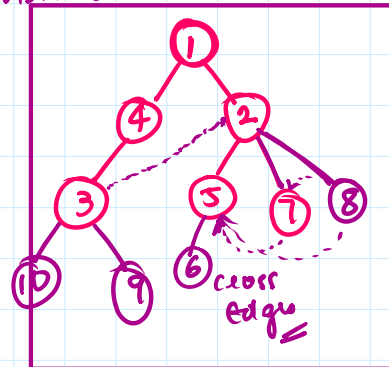
Stack D.A.

while (!q.empty())

Graph



visited



BFS - spanning tree

→ queue: empty
 → 1 4 2 3 5 7 8 10 9 6

↑ ↑
 next next
 q.front()

[op: 1, 4, 2, 3, 5, 7, 8, 10, 9]

traversal.

visited

pseudocode

void bfs (matrix, result, graph)

Adj. matrix, function.

```
void bfs (matrix, result, graph)
```

Adj.-matrix, function.

```

    queue <int> q;  queue
    unordered_map<int, bool> visited;

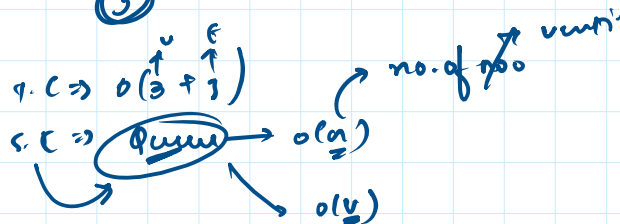
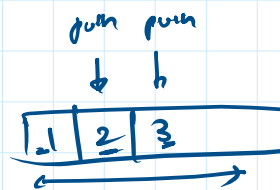
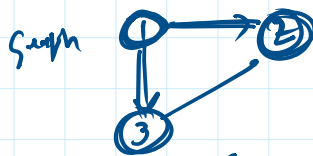
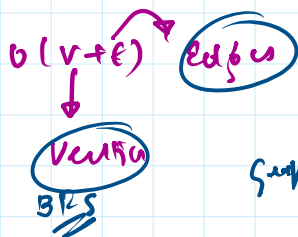
```

```
for(int i; adjMatrix[?][?])
```

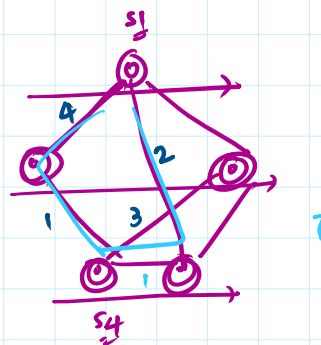
$\delta_i =$ (whether you are visited or not)

finding the Adj-node & traversing it one by one.

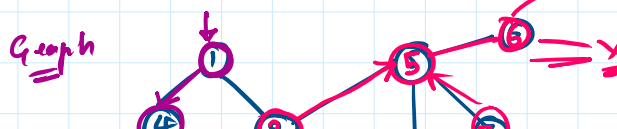
$\mathcal{I}(\text{visited item})$
// nothing



Network → store pattern
network topology

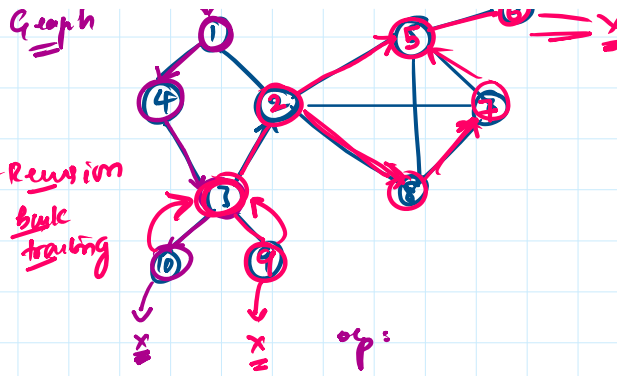


version traversal completed

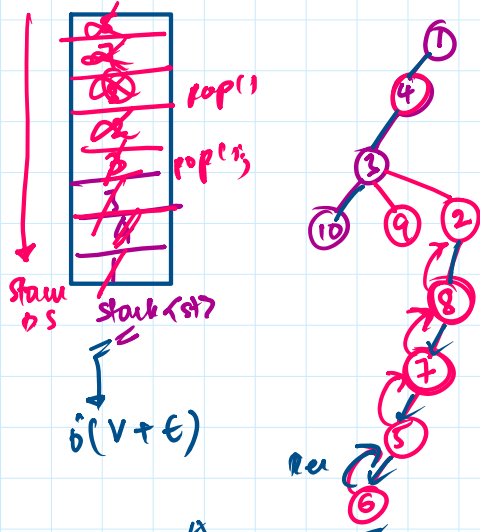


DFS → Depth First Search:

DFS → Depth first search:
 DS: Stack:
 ↳ linear DS



op: 1, 4, 3, 10, 9, 2, 8, 7, 5, 6
 → DFS &



DFS spanning tree:
 1. C → $O(V+E)$
 1. D →

① Algorithms

- ↳ Topological sort
- ↳ Dijkstra's Algo
- ↳ Prim's Algo, Kruskal's Algo
- ↳ Huffman's coding-decoding
- ↳ Leaden's Algo.

1. C, S-C, Algo courses, Appli
 cation
 → interview
 understanding

DP: Dynamic programming + recursion on recursive pattern